COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	10/01/2026	**DATE SUBMITTED:**	10/01/2026
NAME:	KEITH NICOLE SILVOZA	**INSTRUCTOR:**	ENGR. JIMLORD M. QUEJADO

ACTIVITY NO. 10.2 Graph

1. PROCEDURE OUTPUT

What procedures did you perform?

1. Define the following: (10)
   A. Graph
   B. Vertex
   C. Nodes
   D. Weight
   E. Path
   F. Directed graph
   G. Connected graph
   H. Directed cyclic
   I. Adjacency matrix
   J. Adjacency List

GRAPH - A NON-LINEAR DATA STRUCTURE CONSISTING OF A FINITE SET OF VERTICES (OR NODES) AND A SET OF EDGES CONNECTING PAIRS OF VERTICES.
VERTEX- A FUNDAMENTAL UNIT OR POINT IN A GRAPH REPRESENTING AN ENTITY (E.G., A CITY, A COMPUTER, OR A WEB PAGE)
NODES- SYNONYM FOR VERTICES; INDIVIDUAL ELEMENTS OR DATA POINTS WITHIN A NETWORK/GRAPH STRUCTURE.
WEIGHT- A NUMERICAL VALUE ASSIGNED TO AN EDGE IN A WEIGHTED GRAPH REPRESENTING A COST, DISTANCE, CAPACITY, OR TIME BETWEEN TWO VERTICES.
PATH- A SEQUENCE OF ALTERNATING VERTICES AND EDGES STARTING AT A SOURCE VERTEX AND ENDING AT A TARGET VERTEX, WHERE EACH EDGE CONNECTS ITS ADJACENT VERTICES.
DIRECTED GRAPH (DIAGRAPH)- A GRAPH WHERE EVERY EDGE HAS A SPECIFIC DIRECTION, INDICATING A ONE-WAY RELATIONSHIP FROM AN ORIGIN VERTEX TO A DESTINATION VERTEX
CONNECTED GRAPH- AN UNDIRECTED GRAPH IN WHICH THERE IS A PATH BETWEEN EVERY PAIR OF VERTICES (NO VERTEX IS COMPLETELY ISOLATED)
DIRECTED CYCLIC GRAPH- A DIRECTED GRAPH CONTAINING AT LEAST ONE CYCLE—A DIRECTED PATH THAT STARTS AND ENDS AT THE SAME VERTEX
ADJACENCY MATRIX- A 2D SQUARE MATRIX OF SIZE $V \times V$ (WHERE V IS THE NUMBER OF VERTICES) USED TO REPRESENT A GRAPH, WHERE CELL $A[i][j]$ INDICATES WHETHER AN EDGE EXISTS BETWEEN VERTEX i AND VERTEX j (AND HOLDS THE EDGE WEIGHT IF APPLICABLE)
ADJACENCY LIST- A GRAPH REPRESENTATION WHERE EACH VERTEX MAINTAINS AN ARRAY OR LINKED LIST OF ALL ITS NEIGHBORING VERTICES (OR CONNECTED EDGES)

2.
EDGE FROM A TO B WITH WEIGHT 3 : (A -> B, w = 3)
EDGE FROM A TO C WITH WEIGHT 2 : (A -> C, w = 2)
EDGE FROM C TO B WITH WEIGHT 1 : (C -> B, w = 1)
EDGE FROM D TO A WITH WEIGHT 4 : (D -> A, w = 4)

3.

SOURCE / DEST	A	B	C	D
A	0	3	2	0
B	0	0	0	0
C	0	1	0	0
D	4	0	0	0

4.
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;
// Structure to represent a weighted edge destination
struct Edge {
char dest;
int weight;
};
class Graph {
unordered_map<char, vector<Edge>> adjList;
public:
void addEdge(char src, char dest, int weight) {
adjList[src].push_back({dest, weight});
}
void printGraph() {
vector<char> nodes = {'A', 'B', 'C', 'D'};

for (char node : nodes) {
cout << "Node " << node << " -> ";
if (adjList[node].empty()) {
cout << "No outgoing edges";
} else {
for (const auto& edge : adjList[node]) {
cout << "[" << edge.dest << " | Weight: " << edge.weight << "]
";
}
}
cout << "\n";
}

```
}
};
int main() {
Graph g;
// Adding directed weighted edges according to the figure
g.addEdge(&#39;A&#39;, &#39;B&#39;, 3);
g.addEdge(&#39;A&#39;, &#39;C&#39;, 2);
g.addEdge(&#39;C&#39;, &#39;B&#39;, 1);
g.addEdge(&#39;D&#39;, &#39;A&#39;, 4);
cout &lt;&lt; &quot;Adjacency List Representation of the Graph:\n\n&quot;;
g.printGraph();
return 0;
}
```

5.

FEATURE / ASPECT	BREADTH-FIRST SEARCH (BFS)	DEPTH-FIRST SEARCH (DFS)
TRAVERSAL STRATEGY	EXPLORES GRAPH LEVEL-BY-LEVEL (VISITS ALL IMMEDIATE NEIGHBORS BEFORE MOVING DEEPER)	EXPLORES AS DEEP AS POSSIBLE ALONG EACH BRANCH BEFORE BACKTRACKIN
DATA STRUCTURE USED	QUEUE (FIFO - FIRST-IN, FIRST-OUT)	STACK (LIFO - LAST-IN, FIRST-OUT) OR CALL STACK (RECURSION)
SHORTEST PATH GUARANTEE	GUARANTEES FINDING THE SHORTEST PATH ON UNWEIGHTED GRAPHS	DOES NOT GUARANTEE FINDING THE SHORTEST PATH
MEMORY COMPLEXITY	HIGHER MEMORY USAGE (O(V)) AS IT STORES ALL NODES AT THE CURRENT DEPTH	GENERALLY LOWER MEMORY FOOTPRINT (O(H), WHERE H IS THE TREE/GRAPH HEIGHT)
PRIMARY APPLICATIONS	FINDING SHORTEST PATH, PEER-TO-PEER NETWORK ROUTING, WEB CRAWLERS	

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.